BasicTranslator

October 22, 2026

Por: José Adrián Rodríguez González y Esmeralda Abigail Ruiz Vazquez

1 NMT Tarea

Para esta tarea se va a realizar un modelo en el que se busca traducir. El primer paso que se realiza es una lectura inicial del conjunto de datos que proviene de https://huggingface.co/datasets/Helsinki-NLP/opus-100 Este conjunto contiene varios idiomas y es un clasico de traduccion

2 1 EDA

2.1 1.1 Lectura

Para el EDA inicial, se observara el tamaño, la existencia de caracteres nulos, duplicados exactos La siguiente seccion, lee el conjunto de datos. En este caso, se muestra el conjunto de datos y una muestra

[1]:
```
from datasets import load_dataset
corpus= load_dataset("Helsinki-NLP/opus-100", "en-es")
```

```
Warning: You are sending unauthenticated requests to the HF Hub. Please set a
HF_TOKEN to enable higher rate limits and faster downloads.
```

[2]:
```
print(corpus)
print(type(corpus))
print(f"Lista de llaves{list(corpus.keys())}")
for split,data in corpus.items():
    print(f"{split}: {len(data):,} filas")
    print(type(data))
    print("Columnas:", data.column_names)
example = corpus["train"][0]

print("English:", example["translation"]["en"])
print("Spanish:", example["translation"]["es"])
```

```
DatasetDict({
    test: Dataset({
        features: ['translation'],
        num_rows: 2000
    })
```

```
    train: Dataset({
        features: ['translation'],
        num_rows: 1000000
    })
    validation: Dataset({
        features: ['translation'],
        num_rows: 2000
    })
})
<class 'datasets.dataset_dict.DatasetDict'>
Lista de llaves['test', 'train', 'validation']
test: 2,000 filas
<class 'datasets.arrow_dataset.Dataset'>
Columnas: ['translation']
train: 1,000,000 filas
<class 'datasets.arrow_dataset.Dataset'>
Columnas: ['translation']
validation: 2,000 filas
<class 'datasets.arrow_dataset.Dataset'>
Columnas: ['translation']
English: It was the asbestos in here, that's what did it!
Spanish: Fueron los asbestos aquí. ¡Eso es lo que ocurrió!
```

Como se puede notar, el conjunto esta en un formato de json, y se interpreta como un diccionario de python, que tiene las claves de los splits definidos, como test,train y validation. EN cada uno tiene de columnas una traduccion y es un objeto de tipo arrow_dataset.Dataset. Esta es una facilidad que tiene la libreria `dataset` ya que nos permite descargar conjuntos de datos de hugging face y ejecutarlos de manera local ## 1.2 Crear dataframe de analisis Por facilidad, vamos a utilizar un dataframe de analisis para poder ser utilizado en los posteriores calculos

[3]:
```
import numpy
import pandas as pd
def to_dataframe(split):
    translations=corpus[split]["translation"]
    return pd.DataFrame({
        "en":[pair["en"] for pair in translations],
        "es":[pair["es"] for pair in translations]
             })
train_df=to_dataframe("train")
val_df=to_dataframe("validation")
test_df=to_dataframe("test")
```

[4]:
```
print("Train:", train_df.shape)
print("Validation:", val_df.shape)
print("Test:", test_df.shape)
```

```
Train: (1000000, 2)
Validation: (2000, 2)
Test: (2000, 2)
```

Observamos en la tabla las dimensiones de los conjuntos de datos | split| filas | columnas | |——|——-|———-| |train | 1,000,000| 2| |Validation | 2,000| 2| |test | 2,000| 2|

2.2 1.3 Seccion de EDA

Se realiza un pequeño analisis exploratorio de los datos para obsservar la distribucion del conjunto de datos

[5]:
```
def basic_eda(df, name):
    print(f"\n{'=' * 50}")
    print(f"EDA: {name}")
    print(f"{'=' * 50}")
    print(f"Dimensiones: {df.shape}")
    print("\nNulos:")
    print(df.isna().sum())
    print("\nCadenas vacías:")
    print(df[["en", "es"]].apply(lambda col: col.str.strip().eq("")).sum())
    print("\nPares duplicados:", df.duplicated(["en", "es"]).sum())
    print("\nEstadísticas de palabras:")
    for lang in ["en", "es"]:
        lengths = df[lang].str.split().str.len()
        print(f"\n{lang}:")
        print(lengths.describe(percentiles=[0.50, 0.75, 0.90, 0.95, 0.99]))
for name, df in [
    ("Train", train_df),
    ("Validation", val_df),
    ("Test", test_df),
]:
    basic_eda(df, name)
```

```
==================================================
EDA: Train
==================================================
Dimensiones: (1000000, 2)

Nulos:
en    0
es    0
dtype: int64

Cadenas vacías:
en    0
es    0
dtype: int64

Pares duplicados: 70318
```

```
Estadísticas de palabras:

en:
count    1000000.000000
mean          10.753217
std           21.950766
min            1.000000
50%            7.000000
75%           13.000000
90%           25.000000
95%           35.000000
99%           58.000000
max        18016.000000
Name: en, dtype: float64

es:
count    1000000.000000
mean          11.076716
std           19.346420
min            1.000000
50%            6.000000
75%           13.000000
90%           27.000000
95%           39.000000
99%           66.000000
max        10204.000000
Name: es, dtype: float64

==================================================
EDA: Validation
==================================================
Dimensiones: (2000, 2)

Nulos:
en    0
es    0
dtype: int64

Cadenas vacías:
en    0
es    0
dtype: int64

Pares duplicados: 2

Estadísticas de palabras:

en:
```

```
count    2000.000000
mean       12.723500
std        12.634115
min         1.000000
50%         8.000000
75%        16.000000
90%        29.000000
95%        38.000000
99%        61.000000
max       172.000000
Name: en, dtype: float64

es:
count    2000.000000
mean       13.381000
std        14.823856
min         1.000000
50%         8.000000
75%        16.000000
90%        33.000000
95%        42.000000
99%        68.020000
max       222.000000
Name: es, dtype: float64

==================================================
EDA: Test
==================================================
Dimensiones: (2000, 2)

Nulos:
en    0
es    0
dtype: int64

Cadenas vacías:
en    0
es    0
dtype: int64

Pares duplicados: 1

Estadísticas de palabras:

en:
count    2000.000000
mean       12.721500
std        12.268096
```

```
min          1.000000
50%          9.000000
75%         16.000000
90%         28.000000
95%         38.000000
99%         58.000000
max        137.000000
Name: en, dtype: float64

es:
count    2000.000000
mean       13.322000
std        14.412015
min         1.000000
50%         8.000000
75%        16.000000
90%        32.000000
95%        43.000000
99%        66.000000
max       156.000000
Name: es, dtype: float64
```

[6]:
```python
def inspect_longest_pairs(df, lang="en", n=10):
    # Adaptamos los nombres a los que aparecen en tu salida: en y es.
    col = lang

    result = df.loc[
        df[col].str.split().str.len().nlargest(n).index
    ].copy()

    result["en_words"] = result["en"].str.split().str.len()
    result["es_words"] = result["es"].str.split().str.len()

    return result[
        ["en", "es", "en_words", "es_words"]
    ].sort_values("en_words", ascending=False)

inspect_longest_pairs(train_df, "en", 10)
```

[6]:
```
                                                       en  \
399114  Fifty-ninth session * A/59/50 and Corr.1. Item…
905256  ==Dictis=="Dictis" L. Koch, 1872* "Dictis dent…
883727  Open-ended Working Group of the Basel Conventi…
557642  Ratify human rights treaties to which Tuvalu i…
616955  ==Afrofilistata=="Afrofilistata" Benoit, 1968*…
350997  ==Filmography==* "The Quagmire" (1916)* "The R…
4326     Thula Mama, Thula Mama, Thula Mama, Thula Ma…
```

```
711484  By resolution 1305 (2000), the Security Counci…
187838  Sacred bell of King Seongdeok the Great, Gyeon…
265025  * 1952 - São Paulo SP - MAM/SP* 1954 - Salvado…
```

	es	en_words	es_words
399114	Quincuagésimo noveno período de sesiones Tema …	18016	75
905256	==Dictis=="Dictis" L. Koch, 1872* "Dictis stri…	1737	1721
883727	Designación de autoridades competentes y punto…	1293	1163
557642	Intensificar las actividades de sensibilizació…	871	174
616955	==Afrofilistata=="Afrofilistata" Benoit, 1968*…	853	848
350997	== Filmografía ==* "The Quagmire" (1916)* "The…	614	622
4326	Thula Mama, Thula Mama, Thula Mama, Thula Ma…	609	551
711484	En su resolución 1305 (2000), el Consejo de Se…	599	629
187838	Campana Sagrado de Rey Seongdeok Magno, Museo …	597	663
265025	) - en el Instituto de Artes de Filadelfia; Mé…	530	301

[7]:
```python
def add_quality_features(df):
    df = df.copy()

    for lang in ["en", "es"]:
        df[f"{lang}_words"] = df[lang].str.split().str.len()
        df[f"{lang}_chars"] = df[lang].str.len()

    df["word_ratio"] = (
        (df["en_words"] + 1) / (df["es_words"] + 1)
    )

    df["word_diff"] = (
        df["en_words"] - df["es_words"]
    ).abs()

    df["max_words"] = df[["en_words", "es_words"]].max(axis=1)

    return df


train_analysis = add_quality_features(train_df)


thresholds = [64, 128, 256, 512]

for limit in thresholds:
    mask = (
        (train_analysis["en_words"] > limit)
        | (train_analysis["es_words"] > limit)
    )
```

```
    count = int(mask.sum())

    print(
        f"> {limit:>3} palabras: "
        f"{count:>8,} pares "
        f"({count / len(train_analysis) * 100:.3f}%)"
    )
ratio_mask = (
    (train_analysis["word_ratio"] > 3)
    | (train_analysis["word_ratio"] < 1 / 3)
)

print("Pares con razón extrema:", ratio_mask.sum())
print(
    "Porcentaje:",
    f"{ratio_mask.mean() * 100:.3f}%"
)
```

```
>  64 palabras:   11,569 pares (1.157%)
> 128 palabras:      693 pares (0.069%)
> 256 palabras:       63 pares (0.006%)
> 512 palabras:       15 pares (0.002%)
Pares con razón extrema: 5484
Porcentaje: 0.548%
```

Con este analisis previo, se encontró que existen varios elementos duplicados en el conjunto datos, sin embargo, el hallazgo mas interesante fue ver la diferencias fuertes de tokens que hay entre unos cuantos pares, pero a su vez, que es el problema mas fuerte, hay varias muestras que ocntiene una gran cantidad de tokens y se ve que son elementos muy largos o documentos.

2.3 1.4 Limpieza del conjunto de datos

EN la limpieza de conjunto de datos, se quitaron los documentos duplicado y posibles documentos que puedan ser un leakage para nuestros datos

[8]:
```
def duplicate_report(df, name):
    duplicated = df.duplicated(subset=["en", "es"])
    print(f"\n{name}")
    print(f"Total de pares: {len(df):,}")
    print(f"Filas duplicadas: {duplicated.sum():,}")
    print(f"Pares únicos: {df.drop_duplicates(['en', 'es']).shape[0]:,}")
    print(f"Porcentaje duplicado: {duplicated.mean() * 100:.3f}%")
def pair_set(df):
    return set(zip(df["en"], df["es"]))


for name, df in [("Train", train_df),("Validation", val_df),("Test", test_df)]:
    duplicate_report(df, name)
```

```
train_pairs = pair_set(train_df)
validation_pairs = pair_set(val_df)
test_pairs = pair_set(test_df)

print("\nTrain  Validation:", len(train_pairs & validation_pairs))
print("Train  Test:", len(train_pairs & test_pairs))
print("Validation  Test:", len(validation_pairs & test_pairs))


###aplicamos la limpiea
train_clean_df=(train_df.drop_duplicates(subset=["en","es"]).
 ↪reset_index(drop=True))
validation_clean_df=(val_df.drop_duplicates(subset=["en","es"]).
 ↪reset_index(drop=True))
test_clean_df=(test_df.drop_duplicates(subset=["en","es"]).
 ↪reset_index(drop=True))

validation_unique_pairs=set(zip(validation_clean_df["en"],validation_clean_df["es"]))
test_cleaned_df=(test_clean_df.loc[~test_clean_df.apply(lambda row:␣
 ↪(row["en"],row["es"]) in validation_unique_pairs,axis=1 )].
 ↪reset_index(drop=True))

## verficiacion de limpieaz
for name, df in [
    ("\nTrain", train_clean_df),
    ("Validation", validation_clean_df),
    ("Test", test_clean_df),
]:
    print(
        f"{name}: {len(df):,} filas | "
        f"{df.duplicated(['en', 'es']).sum():,} duplicados"
    )

train_pairs = set(zip(train_clean_df["en"], train_clean_df["es"]))
validation_pairs = set(zip(validation_clean_df["en"],␣
 ↪validation_clean_df["es"]))
test_pairs = set(zip(test_cleaned_df["en"], test_clean_df["es"]))

print("Train  Validation:", len(train_pairs & validation_pairs))
print("Train  Test:", len(train_pairs & test_pairs))
print("Validation  Test:", len(validation_pairs & test_pairs))
```

```
Train
Total de pares: 1,000,000
```

```
Filas duplicadas: 70,318
Pares únicos: 929,682
Porcentaje duplicado: 7.032%

Validation
Total de pares: 2,000
Filas duplicadas: 2
Pares únicos: 1,998
Porcentaje duplicado: 0.100%

Test
Total de pares: 2,000
Filas duplicadas: 1
Pares únicos: 1,999
Porcentaje duplicado: 0.050%

Train   Validation: 0
Train   Test: 0
Validation   Test: 3

Train: 929,682 filas | 0 duplicados
Validation: 1,998 filas | 0 duplicados
Test: 1,999 filas | 0 duplicados
Train   Validation: 0
Train   Test: 0
Validation   Test: 0
```

3 2 Corpus

Para esta parte es donde se comienza a realizar el desarrollo del formato de corpus para la representacion vectorial postuma. EL primer intento, es hacer que los conjuntos inicialmente cargados en memoria se guarden en un archivo

[9]:
```python
import os

data_dir = os.path.join("data", "processed")
os.makedirs(data_dir, exist_ok=True)

corpora = {
    "train.csv": train_clean_df,
    "validation.csv": validation_clean_df,
    "test.csv": test_cleaned_df,
}

for filename, df in corpora.items():
    df[["en", "es"]].to_csv(os.path.join(data_dir,␣
 ↪filename),index=False,encoding="utf-8",)
```

4 3 Normalizaicon y tokenizacion

Para esta seccion, se intenta limpiar el conjunto de datos con tecncias como - Remover mayusculas - Cuidar acentos - Normalziar uNicode - Reducir espacios - Unificar apostrofes La expresion regular propuesta busca o que haya apostrofes o que sean espacios para poder tokenizer, de esta manera, palabras como “don’t”, no se expanden. COn este regex, con la segunda alternativa despues del |, indica que o es una expresion u otra, y la segunda expresion es la que nos permite tokenizar a nivel palabra

[10]:
```
import re
import unicodedata

TOKEN_PATTERN = re.compile(r"\w+(?:['’]\w+)*|[^\w\s]",flags=re.UNICODE,)
def normalize_text(text):
    text = unicodedata.normalize("NFC", text)
    text = text.replace("’", "'")
    text = text.lower()
    text = re.sub(r"\s+", " ", text).strip()
    return text


def tokenize(text):
    normalized = normalize_text(text)
    return TOKEN_PATTERN.findall(normalized)
def detokenize(tokens):
    """Reconstruye texto a partir de tokens y ajusta la puntuación."""
    text = " ".join(tokens)
    # Quitar espacios antes de signos de puntuación.
    text = re.sub(r"\s+([.,!?;:%)\]}])", r"\1", text)
    # Quitar espacios después de signos de apertura.
    text = re.sub(r"([(\\[{¿¡])\s+", r"\1", text)
    # Reconstruir contracciones inglesas si aparecen.
    text = re.sub(r"\s+(['’])\s*", r"\1", text)
    return text.strip()
```

[11]:
```
def preprocess_corpus(df):
    result = df[["en", "es"]].copy()

    for lang in ("en", "es"):
        result[f"{lang}_norm"] = (
            result[lang].map(normalize_text)
        )
        result[f"{lang}_tokens"] = (
            result[f"{lang}_norm"].map(tokenize)
        )

    return result
```

```
train_processed = preprocess_corpus(train_clean_df)
validation_processed = preprocess_corpus(validation_clean_df)
test_processed = preprocess_corpus(test_cleaned_df)
```

5 3 Vocabulario

Se construye por medio de una clase, un vocabualrio, que contendra la frecuencia y la forma de acceder a cada token

[12]:
```
class Vocabulary:
    def __init__(self):
        self.token_freq= {}
        self.token_to_id= {}
        self.id_to_token={}
        self.unk_token = "<UNK>"
        self.pad_token = "<PAD>"
        self.unk_id = 0
        self.pad_id = 1
    def build(self,sentences):
        self.token_freq = {}
        for tokens in sentences:
            for token in tokens:
                self.token_freq[token]=(self.token_freq.get(token,0)+1)
        sorted_tokens=sorted(self.token_freq,key=lambda token:(-self.
 ↪token_freq[token],token))
        special_tokens = [self.unk_token, self.pad_token]
        sorted_tokens = [token for token in sorted_tokens if token not in␣
 ↪special_tokens]
        self.token_to_id= {
            token:idx
            for idx,token in enumerate(special_tokens)
        }
        for token in sorted_tokens:
            self.token_to_id[token] = len(self.token_to_id)
        self.id_to_token={
            idx:token
            for token,idx in self.token_to_id.items()
        }
    def __len__(self):
        return len(self.token_to_id)
    def get_id(self,token):
        return self.token_to_id.get(token,self.unk_id)
    def get_token(self,idx):
        return self.id_to_token.get(idx,self.unk_token)
    def get_frequency(self,token):
```

```
        return self.token_freq.get(token,0)
    def encode(self, tokens):
        return [self.get_id(token) for token in tokens]
    def decode(self, ids):
        return [self.get_token(token_id) for token_id in ids]
    def most_common(self,n=10):
        return sorted(self.token_freq.items(), key=lambda item:
↪(-item[1],item[0]))[:n]
    def report(self):
        total_tokens = sum(self.token_freq.values())
        unique_tokens = len(self.token_freq)
        singletons = sum(
            1 for freq in self.token_freq.values()
            if freq == 1
        )

        return {
            "total_tokens": total_tokens,
            "unique_tokens": unique_tokens,
            "singletons": singletons,
            "singleton_ratio": (
                singletons / unique_tokens
                if unique_tokens else 0.0
            ),
        }
```

[13]:
```
en_vocab = Vocabulary()
es_vocab = Vocabulary()

en_vocab.build(train_processed["en_tokens"])
es_vocab.build(train_processed["es_tokens"])
```

[14]:
```
print("Tamaño del vocabulario inglés:", len(en_vocab))
print("Tamaño del vocabulario español:", len(es_vocab))

print("ID de 'the':", en_vocab.get_id("the"))
print("Token con ID 0:", en_vocab.get_token(0))
print("Frecuencia de 'the':", en_vocab.get_frequency("the"))

print("\nTokens más frecuentes en inglés:")
for token, frequency in en_vocab.most_common(10):
    print(f"{token!r}: {frequency:,}")

print("\nEstadísticas:")
print(en_vocab.report())
```

```
tokens = ["the", "cat", "supercalifragilistic"]

ids = en_vocab.encode(tokens)

print("Tokens:", tokens)
print("IDs:", ids)
print("Recuperación:", en_vocab.decode(ids))
```

```
Tamaño del vocabulario inglés: 185548
Tamaño del vocabulario español: 238130
ID de 'the': 3
Token con ID 0: <UNK>
Frecuencia de 'the': 621242

Tokens más frecuentes en inglés:
'.': 906,498
'the': 621,242
',': 608,786
'of': 321,661
'to': 287,856
'and': 263,510
'a': 197,603
'in': 193,600
'you': 181,251
'-': 180,006

Estadísticas:
{'total_tokens': 12867574, 'unique_tokens': 185546, 'singletons': 102809,
'singleton_ratio': 0.5540890129671349}
Tokens: ['the', 'cat', 'supercalifragilistic']
IDs: [3, 2492, 0]
Recuperación: ['the', 'cat', '<UNK>']
```

[15]:
```
def unknown_report(vocab, df, lang):
    total = 0
    unknown = 0

    for tokens in df[f"{lang}_tokens"]:
        total += len(tokens)
        unknown += sum(
            token not in vocab.token_to_id
            for token in tokens
        )

    return {
        "total_tokens": total,
        "unknown_tokens": unknown,
```

```
        "unknown_rate": unknown / total if total else 0.0,
    }


print("Inglés - validación:", unknown_report(en_vocab, validation_processed,␣
 ↪"en"))

print("Inglés - test:", unknown_report(en_vocab, test_processed, "en"))

print("Español - validación:", unknown_report(es_vocab, validation_processed,␣
 ↪"es"))

print("Español - test:", unknown_report(es_vocab, test_processed, "es"))
```

```
Inglés - validación: {'total_tokens': 30799, 'unknown_tokens': 288,
'unknown_rate': 0.009350952953017955}
Inglés - test: {'total_tokens': 30607, 'unknown_tokens': 236, 'unknown_rate':
0.007710654425458285}
Español - validación: {'total_tokens': 32048, 'unknown_tokens': 305,
'unknown_rate': 0.00951697453819271}
Español - test: {'total_tokens': 31721, 'unknown_tokens': 307, 'unknown_rate':
0.00967813120645629}
```

6 4 Representacion vectorial del vocabulario

Ya que vamos a pasarlo por un conjunto de datos, tenemos que mandarlo a un mundo o representación vectorial, tanto puede ser como TFIDF u otra, en este caso y acorde a la literatura, se usan *matrices de coocurencia*, que lo que buscan es, si tengo una palabra en español ¿cuantas veces aparece en ingles?

Vamos a formular esta matriz de la siguiente forma

$$\mathcal{D} = \{(x^n, y^n)\}_{n=1}^{N}$$

Donde x^n represent el vector de un idioma con sus tokens y el y_n, el otro par del vector. Y N, seria la cantidad total de filas que tiene nuestro dataset o mejor dicho, la cantidad de pares que tengamos en nuestro conjunto de datos. Un ejemplo de esto seria | N | x^n | y^n | |—|——-|——-| |1| The cat is eating| El gato esta comiend| |—|——-|——-|

Como cada oracion, ya fue previamente tokenizada,entonces podemos reescribir cada oración como

$$x^n = (x_1^n, \dots, x_{L_x}^n)$$

$$y^n = (y_1^n, \dots, y_{L_y}^n)$$

Donde el valor L, corresponde al valor máximo de tokens que tenga la frase, sin embargo, ahora que tenemos esta forma, ya tenemos los vocabularios de ambos idiomas como

$$\mathcal{V}_{en} = \{e_1, e_2, \dots, e_{|\mathcal{V}_{en}|}\}$$

$$\mathcal{V}_{es} = \{e_1, e_2, \dots, e_{|\mathcal{V}_{es}|}\}$$

Donde e_1, es cada palabra dentro del vocabulario y $|\mathcal{V}_{ex}|$ es la longitud de un determinado vocabulario. Estos tamaños finales, contemplan los caracteres especiales como `<UNK>` y `<PAD>` ## Definamos la matriz de coocurrencia

Ahora, es cuestión de como definir la matriz, y bajo la propuesta que definimos, diriamos que en una fila iria un idioma, y en las columnas iria el otro idioma, matematicamente esto se escribiria como

$$C \in \mathbb{N}^{\mathcal{V}_{en} \times \mathcal{V}_{es}}$$

Ahora, la idea es calcular la aparicion de una palabra en una frase, asi que

$$f(e_i, x^n) = \begin{cases} 1 & e_i \in x^n \\ 0 & e_i \notin x^n \end{cases}$$

Esto mismo, pasaria para el otro vocabulario, de tal manera, que la matriz se construye como

$$C_{ij} = \sum_{n=1}^{N} f(en_i, x^n) f(es_j, y^n)$$

Finalmente, podemos obtener una probabildiad

$$P(e_s|e_n) = \frac{C(e_n, e_s)}{\sum_{e'_s} C(e_n, e'_s)}$$

Con esta probabildiad, rpacticamente tenemos un mini traductor simple, que lo que dice es "que probabildiad tenemos de que un token en español sea dado a un token en ingles". Es decir, "que palabra posiblemente se asocia el ingles al español", y el calculo corresponde con la coocurrencia del token ingles y español entre la suma de ese token en ingles con todos los de español

[16]:
```python
import math
import numpy as np
from scipy.sparse import csr_matrix

class BilingualCooccurrenceMatrix:
    def __init__(self, source_vocab, target_vocab):
        self.source_vocab = source_vocab
        self.target_vocab = target_vocab

        self.matrix = {}
        self.source_totals = {}
        self.target_totals = {}
        self.num_pairs = 0

    def fit(self, source_sentences, target_sentences):
        if len(source_sentences) != len(target_sentences):
```

```
            raise ValueError("Los corpus deben tener la misma cantidad de␣
↪oraciones.")
        self.matrix = {}
        self.source_totals = {}
        self.target_totals = {}
        self.num_pairs = len(source_sentences)
        for source_tokens, target_tokens in zip(source_sentences,␣
↪target_sentences):
            source_unique = set(source_tokens)
            target_unique = set(target_tokens)
            source_ids = {
                self.source_vocab.get_id(token)
                for token in source_unique
            }
            target_ids = {
                self.target_vocab.get_id(token)
                for token in target_unique
            }
            source_ids.discard(self.source_vocab.unk_id)
            source_ids.discard(self.source_vocab.pad_id)
            target_ids.discard(self.target_vocab.unk_id)
            target_ids.discard(self.target_vocab.pad_id)
            for source_id in source_ids:
                self.source_totals[source_id] = (self.source_totals.
↪get(source_id, 0) + 1)
                row = self.matrix.setdefault(source_id, {})
                for target_id in target_ids:
                    row[target_id] = row.get(target_id, 0) + 1
            for target_id in target_ids:
                self.target_totals[target_id] = (self.target_totals.
↪get(target_id, 0) + 1)
        return self

    def get_count(self, source_token, target_token):
        source_id = self.source_vocab.get_id(source_token)
        target_id = self.target_vocab.get_id(target_token)
        if source_id in (self.source_vocab.unk_id,self.source_vocab.pad_id,):
            return 0
        if target_id in (self.target_vocab.unk_id,self.target_vocab.pad_id):
            return 0

        return self.matrix.get(source_id, {}).get(target_id, 0)
    def get_translation_candidates(self, source_token, n=10):
        source_id = self.source_vocab.get_id(source_token)
        if source_id in (self.source_vocab.unk_id,self.source_vocab.pad_id,):
            return []
        row = self.matrix.get(source_id, {})
```

```
        candidates = sorted(row.items(),key=lambda item: (-item[1], item[0]),)[:
↪n]
        return [
            (self.target_vocab.get_token(target_id), count)
            for target_id, count in candidates
        ]

    def get_probability(self, source_token, target_token):
        source_id = self.source_vocab.get_id(source_token)
        target_id = self.target_vocab.get_id(target_token)
        if source_id in (self.source_vocab.unk_id,self.source_vocab.pad_id,) or␣
↪target_id in (self.target_vocab.unk_id,self.target_vocab.pad_id,):
            return 0.0
        row = self.matrix.get(source_id, {})
        total = sum(row.values())
        if total == 0:
            return 0.0
        return row.get(target_id, 0) / total

    def num_observed_pairs(self):
        return sum(len(row) for row in self.matrix.values())

    def get_pmi(self, source_token, target_token):
        source_id = self.source_vocab.get_id(source_token)
        target_id = self.target_vocab.get_id(target_token)
        if source_id in (self.source_vocab.unk_id,self.source_vocab.pad_id,) or␣
↪target_id in (self.target_vocab.unk_id,self.target_vocab.pad_id,):
            return 0.0
        cooccurrence = self.matrix.get(source_id, {}).get(target_id, 0)
        source_count = self.source_totals.get(source_id, 0)
        target_count = self.target_totals.get(target_id, 0)
        if (cooccurrence == 0 or source_count == 0 or target_count == 0):
            return 0.0
        return math.log((cooccurrence * self.num_pairs)/ (source_count *␣
↪target_count))
    def get_ppmi(self, source_token, target_token):
        return max(0.0, self.get_pmi(source_token, target_token))

    def get_ppmi_candidates(self, source_token, n=10, min_count=10):
        source_id = self.source_vocab.get_id(source_token)
        if source_id in (self.source_vocab.unk_id,self.source_vocab.pad_id,):
            return []
        row = self.matrix.get(source_id, {})
        candidates = []
        source_count = self.source_totals.get(source_id, 0)
        for target_id, count in row.items():
            if count < min_count:
```

```
                continue
            target_count = self.target_totals.get(target_id, 0)
            if source_count == 0 or target_count == 0:
                continue
            pmi = math.log((count * self.num_pairs)/ (source_count *␣
↪target_count))
            ppmi = max(0.0, pmi)
            candidates.append((target_id, ppmi, count))
        candidates.sort(key=lambda item: (-item[1], item[0]))
        return [
            (self.target_vocab.get_token(target_id), score, count)
            for target_id, score, count in candidates[:n]
        ]

    def build_ppmi_matrix(self, min_count=200):
        source_ids = sorted(self.matrix.keys())
        target_ids = sorted(self.target_totals.keys())
        source_index = {
            token_id: i for i, token_id in enumerate(source_ids)
        }
        target_index = {
            token_id: i for i, token_id in enumerate(target_ids)
        }
        rows, cols, values = [], [], []
        N = self.num_pairs
        if N == 0:
            raise ValueError("La matriz no ha sido entrenada.")
        for source_id, row in self.matrix.items():
            source_count = self.source_totals.get(source_id, 0)
            if source_count == 0:
                continue
            for target_id, count in row.items():
                target_count = self.target_totals.get(target_id, 0)
                if count < min_count:
                    continue
                if count == 0 or target_count == 0:
                    continue
                pmi = np.log((count * N) / (source_count * target_count))
                if pmi > 0:
                    rows.append(source_index[source_id])
                    cols.append(target_index[target_id])
                    values.append(pmi)
        X = csr_matrix(
            (values, (rows, cols)),
            shape=(len(source_ids), len(target_ids)),
            dtype=np.float32,
        )
```

```
        return X, source_ids, target_ids
```

[18]:
```
cooc_matrix_es_en = BilingualCooccurrenceMatrix(es_vocab, en_vocab)
# Primero probamos con 1,000 pares.
cooc_matrix_es_en.fit(train_processed["es_tokens"],train_processed["en_tokens"])
print("ID de gato en español:", es_vocab.get_id("gato"))
print("ID de cat en inglés:", en_vocab.get_id("cat"))

print("UNK español:", es_vocab.unk_id)
print("UNK inglés:", en_vocab.unk_id)

print("¿Existe gato?", es_vocab.get_id("gato") != es_vocab.unk_id)
print("¿Existe cat?", en_vocab.get_id("cat") != en_vocab.unk_id)
print("Pares de oraciones:", cooc_matrix_es_en.num_pairs)
print("Asociaciones distintas:", cooc_matrix_es_en.num_observed_pairs())
print("Coocurrencia entre 'gato' y 'cat':",cooc_matrix_es_en.get_count("gato",␣
 ↪"cat"))
print("Candidatos para 'gato':",cooc_matrix_es_en.
 ↪get_translation_candidates("gato", n=10))
print("Candidatos PPMI para 'gato':",cooc_matrix_es_en.
 ↪get_ppmi_candidates("gato", n=10))
```

```
ID de gato en español: 3636
ID de cat en inglés: 2492
UNK español: 0
UNK inglés: 0
¿Existe gato? True
¿Existe cat? True
Pares de oraciones: 929682
Asociaciones distintas: 44591521
Coocurrencia entre 'gato' y 'cat': 217
Candidatos para 'gato': [('cat', 217), ('.', 215), (',', 119), ('the', 116),
('a', 95), ('and', 59), ('i', 47), ('to', 45), ('of', 43), ('-', 42)]
Candidatos PPMI para 'gato': [('cat', 7.501917871285606, 217), ('dog',
3.8029011285769285, 11), ('like', 1.1384775588618246, 21), ('when',
1.005667362933678, 13), ('my', 0.9116911476643754, 25), ('they',
0.8273595292977217, 16), ('a', 0.7005340151170717, 95), ('your',
0.6970043646068772, 21), ('up', 0.6888578181612468, 13), ('"',
0.6650170989399716, 12)]
```

[19]:
```
print(cooc_matrix_es_en.get_translation_candidates("gato", n=10))
print(cooc_matrix_es_en.get_ppmi_candidates("gato", n=10))
```

```
[('cat', 217), ('.', 215), (',', 119), ('the', 116), ('a', 95), ('and', 59),
('i', 47), ('to', 45), ('of', 43), ('-', 42)]
[('cat', 7.501917871285606, 217), ('dog', 3.8029011285769285, 11), ('like',
1.1384775588618246, 21), ('when', 1.005667362933678, 13), ('my',
```

```
0.9116911476643754, 25), ('they', 0.8273595292977217, 16), ('a',
0.7005340151170717, 95), ('your', 0.6970043646068772, 21), ('up',
0.6888578181612468, 13), ('"', 0.6650170989399716, 12)]
```

[20]:
```
print("Candidatos para 'casa':",cooc_matrix_es_en.
 ↪get_translation_candidates("casa", n=10))
print(cooc_matrix_es_en.get_ppmi_candidates("agua", n=5, min_count=200))

gold_translations_es_en = {
    "gato": {"cat"},
    "perro": {"dog"},
    "casa": {"house"},
    "agua": {"water"},
    "libro": {"book"},
    "comida": {"food"},
    "coche": {"car"},
    "ciudad": {"city"},
    "escuela": {"school"},
    "amigo": {"friend"},
}

for es_word, gold_words in gold_translations_es_en.items():
    candidates = cooc_matrix_es_en.
 ↪get_ppmi_candidates(es_word,n=10,min_count=200,)
    ranked_words = [word for word, score, count in candidates]
    rank = next((i + 1 for i, word in enumerate(ranked_words)if word in␣
 ↪gold_words),None)
    print(f"{es_word:>10} | rango={rank} |  {ranked_words[:5]}")
```

```
Candidatos para 'casa': [('.', 5611), (',', 3188), ('home', 3002), ('the',
2830), ('house', 2715), ('to', 2173), ('and', 1859), ('a', 1589), ('in', 1584),
('you', 1565)]
[('water', 5.576367503875123, 2418), ('or', 1.1309491784143022, 268), ('from',
0.9698613182840673, 261), ('and', 0.923742424323165, 1388), ('(',
0.8894630417316391, 329)]
      gato | rango=1 |  ['cat', '.']
     perro | rango=1 |  ['dog', 'a', 'the', ',', '.']
      casa | rango=1 |  ['house', 'home', 'place', 'go', 'at']
      agua | rango=1 |  ['water', 'or', 'from', 'and', '(']
     libro | rango=1 |  ['book', 'in', 'this', 'a', 'on']
    comida | rango=1 |  ['food', 'and', 'a', 'the', 'you']
     coche | rango=1 |  ['car', 'in', 'a', 'the', 'is']
    ciudad | rango=1 |  ['city', 'town', 'from', 'in', 'which']
   escuela | rango=1 |  ['school', 'at', 'in', 'and', 'to']
     amigo | rango=1 |  ['friend', 'buddy', 'man', 'my', 'your']
```

7 5 Notas

Este metodo que fue, anteriormente aplicado, seria la primera linea base de traduccion, ya que sin necesidad de un modelo, compljeo, sino de probaibldiad bayesiana, podemos encontrar las traducciones de forma simple. Como se notara, la formulacion bayesiana se plantea de diferente manera ya que ahora no es querer intentar saber caul es la siguiente palabra o que la palabara ete asociada a cierta clase, sino que cierta palabra de un vocabulario, este asociado a cierta palabra de otro vocabulario.

Apartir de ahora, se probaran metodos alternos que intentaran optimizar la busqueda y uno de ellos, sera, generar una matriz de ppmi y reducir su dimensionalidad.

8 6 SVD proposicion

EN este caso, conviene utilizar SVD sobre PCA, debido a que estamos manejando matrices disperas, y para fines practicos y optimos, SVD es mas manejable a muy altas dimensionas con valores dispersos. Sabiendo que ls filas representan las palabras en ingles y las columnas las palabras en español, entonces, la manera de obtener los elementos por medio de SVD, es tener en cuenta que al descomposición espectral que hace es

$$X = U\Sigma V^T$$

Donde U, tiene las direcciones latentes de las palabras inglesas, la V^T, del español y Σ los valores singulares ordenados del menor al mayor

A manera de obtener estas representacion,decimos que

$$En = U\Sigma^{1/2}$$

$$Es = V\Sigma^{1/2}$$

De tal forma que ya no tomamos n, sino el valor k, de numero de componentes. Al hacer $1/2$, hacemos una representacion simetrica de la matriz de valores singulares y def esa manera, obtenemos ambos embeddings

[21]:
```
import numpy as np
from pathlib import Path
from sklearn.decomposition import TruncatedSVD
X_es_en, es_ids, en_ids = (cooc_matrix_es_en.build_ppmi_matrix(min_count=200))
print("Dimensiones de PPMI:", X_es_en.shape)
print("Asociaciones positivas:", X_es_en.nnz)
k = 1000
svd =␣
 ↪TruncatedSVD(n_components=k,algorithm="randomized",n_iter=7,random_state=42)
es_embeddings = svd.fit_transform(X_es_en)
sqrt_s = np.sqrt(svd.singular_values_)
es_embeddings /= sqrt_s[None, :]
en_embeddings = svd.components_.T * sqrt_s[None, :]
print("Vectores españoles:", es_embeddings.shape)
print("Vectores ingleses:", en_embeddings.shape)
```

```
explained_variance = float(svd.explained_variance_ratio_.sum())
total_energy = np.sum(X_es_en.data.astype(np.float64) ** 2)
captured_energy = np.sum(svd.singular_values_.astype(np.float64) ** 2)
energy_ratio = (captured_energy / total_energy if total_energy > 0 else␣
 ↪float("nan"))
print(f"Suma de varianza explicada: {explained_variance:.6f}")
print(f"Energía singular capturada: {energy_ratio:.6%}")
output_dir = Path("artifacts/embeddings_es_en")
output_dir.mkdir(parents=True, exist_ok=True)
np.save(output_dir / "es_embeddings.npy",es_embeddings.astype(np.float32))
np.save(output_dir / "en_embeddings.npy",en_embeddings.astype(np.float32))
np.savez_compressed(output_dir / "metadata.npz",es_ids=np.asarray(es_ids,␣
 ↪dtype=np.int64),en_ids=np.asarray(en_ids, dtype=np.int64),
                    k=np.asarray(k),explained_variance=np.
 ↪asarray(explained_variance),energy_ratio=np.asarray(energy_ratio))

print(f"Embeddings guardados en: {output_dir}")
```

```
Dimensiones de PPMI: (238128, 185546)
Asociaciones positivas: 95663
Vectores españoles: (238128, 1000)
Vectores ingleses: (185546, 1000)
Suma de varianza explicada: 0.775483
Energía singular capturada: 77.601600%
Embeddings guardados en: artifacts/embeddings_es_en
```

8.1 Evaluacion inicial de traduccion

Ahora, se va a evaluar el modelo de svd para observar que traducciones da y de ahi, poder implementar mejoras Entonces, si incialmente teniamos $X_{ij} \in \mathbb{R}^{n \times m}$, siendo n, las palabras del vocablo ingles, mientras que m, del vocablo español y el X_{ij} es el $PPMI$ de la palabra e_{n_i} ingles y e_{s_j} española. Asi que con las dimensiones que obtenemos, $PPMI \in \mathbb{R}^{185546 \times 238128}$ tras haber pasado por el SVD, recordemos que las matrices U, Σ, V,obtiene las siguientes dimensiones

$$U_k \in \mathbb{R}^{n \times k}$$
$$\Sigma_k \in \mathbb{R}^{k \times k}$$
$$V_k \in \mathbb{R}^{m \times k}$$

Donde k es la reduccion que aplicamos, asi que tenemos los dos embeddings $En \in \mathbb{R}^{n \times k}$ y $Es \in \mathbb{R}^{m \times k}$, que representan el ingles y el español respectivamente y se obtiene apartir de la multiplicacion matricial simetrica entre U y la matriz de eigenvalores y V con la matriz de eigenvalores, esto anteriormente explicado.

Hasta el momento, es el resumen general que se hizo de operaciones realizadas despues de haber obtenido la matriz de coocurrencia y de haber calculado el svd

Asi que la pregunta interesante viene aqui y es, *¿Como traducimos?* ## Traduccion Entonces, la traduccion se tiene que ver de la siguiente manera: teniendo los embeddings $En \in \mathbb{R}^{n \times k}$ y $Es \in \mathbb{R}^{m \times k}$. Podemos decir que $\mathbf{e} \in En$ y $\mathbf{s} \in Es$, donde cada vector entonces, se representa

como $\mathbf{e}, \mathbf{s} \in \mathbb{R}^{1\times k}$. Y al tener ambos, misma dimension, podemos operar algebraicamente con ellos sin problemas, y tiene una interpretacion linguistica, ya quye estos vectores provienen de la matriz PPMI, asi que el producto punto $\mathbf{e}_i \cdot \mathbf{s}_j \approx X_{ij}$. que anteriormente estabamos usando. Adicionalmente, otra ventaja interesante, es que ya no solamente podemos utilizar el producto punto como medio de traduccion, sino que podemos emplear la distancia coseno

$$\cos(\mathbf{e}_i, \mathbf{s}_j) = \frac{\mathbf{e}_i \mathbf{s}_j^T}{\|\mathbf{e}_i\|_2 \|\mathbf{s}_j\|_2}$$

Asi que el planteamiento de la traduccion se convierte en: Dado un token en ingles, ¿cual es la probabilidad de que un token en español corresponda a este token?, buscandolo de la siguiente manera

$$s^* = \underset{s_j \in V_{es}}{\operatorname{argmax}} \operatorname{sim}(\mathbf{e}, \mathbf{s}_j)$$

Tambien se va a calcular la cantidad de veces que acierta el modelo con una metrica llamada HIT y MMR, en las siguiente nota se explicara en detalle estas metricas

[22]:
```
import numpy as np
from sklearn.preprocessing import normalize
def␣
 ↪get_topk_translations(source_embedding,target_embedding,source_ids,target_ids,source_vocab,
 ↪
    source_norm = normalize(source_embedding, norm="l2")
    target_norm = normalize(target_embedding, norm="l2")
    source_id_to_row = {
        token_id: i for i, token_id in enumerate(source_ids)
    }
    target_id_to_row = {
        token_id: i for i, token_id in enumerate(target_ids)
    }

    source_id = source_vocab.get_id(source_token)
    if source_id not in source_id_to_row:
        return []
    source_row = source_id_to_row[source_id]
    scores = target_norm @ source_norm[source_row]
    k = min(k, len(scores))
    if k == 0:
        return []
    top_indices = np.argpartition(scores, -k)[-k:]
    top_indices = top_indices[np.argsort(scores[top_indices])[::-1]]
    return [
        (target_vocab.get_token(target_ids[j]),float(scores[j]))
        for j in top_indices
    ]
```

[23]:
```
```

```
def␣
 ↪evaluate_hit(gold_translations,source_embedding,target_embedding,source_ids,target_ids,sour
 ↪5, 10),):
    hits = {k: 0 for k in ks}
    evaluated = 0
    skipped = []
    for source_word, gold_words in gold_translations.items():
        source_id = source_vocab.get_id(source_word)
        if source_id not in set(source_ids):
            skipped.append(source_word)
            continue
        candidates =␣
 ↪get_topk_translations(source_embedding,target_embedding,source_ids,target_ids,source_vocab,
        print(f"\n{source_word} | Traducciones correctas: {gold_words}")
        for rank, (word, score) in enumerate(candidates, start=1):
            marker = " >" if word in gold_words else ""
            print(f"{rank:2}. {word:20} {score:.4f}{marker}")
        if not candidates:
            skipped.append(source_word)
            continue
        ranked_words = [word for word, score in candidates]
        evaluated += 1
        for k in ks:
            top_k_words = set(ranked_words[:k])
            if top_k_words.intersection(gold_words):
                hits[k] += 1
    if evaluated == 0:
        raise ValueError("No hay palabras disponibles para evaluar.")
    results = {
        f"Hit@{k}": hits[k] / evaluated
        for k in ks
    }
    return results, evaluated, skipped
```

[24]:
```
for word in ["gato", "perro", "casa", "agua", "libro"]:
    candidates =␣
 ↪get_topk_translations(es_embeddings,en_embeddings,es_ids,en_ids,es_vocab,en_vocab,word,k=5)
    print(f"{word} → {candidates}")
```

```
gato → [('cat', 0.999791145324707), ('club', 0.1263982206583023), ('j',
0.12439747899770737), ('dark', 0.12437564879655838), ('image',
0.11435414850711823)]
perro → [('dog', 0.9519658088684082), ('couple', 0.31012988090515137), ('a',
0.29150962829589844), ('murder', 0.2802388668060303), ('attack',
0.2718161344528198)]
casa → [('home', 0.9091265797615051), ('house', 0.8734805583953857), ('place',
0.5491120219230652), ('her', 0.33502018451690674), ('go', 0.3168913722038269)]
```

```
agua → [('water', 0.5633985996246338), ('or', 0.5087717175483704), ('from',
0.4195985198020935), ('(', 0.34435558319091797), ('origin',
0.34420347213745117)]
libro → [('book', 0.6944507360458374), ('this', 0.4966602027416229), ('church',
0.4242739677429199), ('commitment', 0.39979636669158936), ('is',
0.3558642566204071)]
```

[27]:
```python
import numpy as np
en_embeddings = np.load("artifacts/embeddings/en_embeddings.npy")
es_embeddings = np.load("artifacts/embeddings/es_embeddings.npy")
metadata = np.load("artifacts/embeddings/metadata.npz")
en_ids = metadata["en_ids"]
es_ids = metadata["es_ids"]
gold_translations = {
    "gato": {"cat"},
    "perro": {"dog"},
    "casa": {"house"},
    "agua": {"water"},
    "libro": {"book"},
}
hit_results, evaluated, skipped = evaluate_hit(gold_translations=gold_translations,source_embedding=es_embeddings,
target_embedding=en_embeddings,source_ids=es_ids,target_ids=en_ids,
source_vocab=es_vocab,target_vocab=en_vocab)
print(f"Palabras evaluadas: {evaluated}")
print(f"Palabras omitidas: {skipped}")
for metric, value in hit_results.items():
    print(f"{metric}: {value:.4f} ({value:.2%})")
```

```
gato | Traducciones correctas: {'cat'}
 1. cat                  0.9998 >
 2. club                 0.1227
 3. j                    0.1195
 4. dark                 0.1189
 5. image                0.1113
 6. limits               0.0985
 7. heat                 0.0977
 8. captain              0.0970
 9. india                0.0943
10. federal              0.0894

perro | Traducciones correctas: {'dog'}
 1. dog                  0.9514 >
 2. couple               0.2754
 3. a                    0.2565
```

```
 4. murder               0.2504
 5. attack               0.2378
 6. gift                 0.2280
 7. message              0.2070
 8. bunch                0.2043
 9. shot                 0.1969
10. hit                  0.1947

casa | Traducciones correctas: {'house'}
 1. home                 0.9087
 2. house                0.8729 >
 3. place                0.5479
 4. her                  0.3332
 5. go                   0.3177
 6. when                 0.3061
 7. come                 0.2772
 8. she                  0.2367
 9. back                 0.2184
10. his                  0.2175

agua | Traducciones correctas: {'water'}
 1. water                0.5640 >
 2. or                   0.5080
 3. from                 0.4187
 4. (                    0.3439
 5. as                   0.3430
 6. origin               0.3423
 7. )                    0.3371
 8. production           0.3361
 9. with                 0.3044
10. and                  0.2857

libro | Traducciones correctas: {'book'}
 1. book                 0.7003 >
 2. this                 0.4860
 3. church               0.4111
 4. commitment           0.3900
 5. is                   0.3472
 6. step                 0.3442
 7. blood                0.3386
 8. a                    0.3365
 9. on                   0.3057
10. strong               0.2957
Palabras evaluadas: 5
Palabras omitidas: []
Hit@1: 0.8000 (80.00%)
Hit@5: 1.0000 (100.00%)
Hit@10: 1.0000 (100.00%)
```

9 Este es nuestro primer modelo

AHora vamos a definirlo en una buena clase, para poder ser reutilizable y poder optimizar algunos aprametros. Adicionalmente puede que convenga tambien gaurdar en cache el vocabulario y para este modelo, unimos las anteriores secciones que definimos del pipeline, en donde tenemos la carga de embeddings y vocabulario, a su vez como la extraccion de IDs y normalizacion vectorial. El obtener de manera exhaustivo los top K candidatos y calcular las metricas HIT y MMR ### Métricas de evaluación: Hit@K y Mean Reciprocal Rank

Sea Q el conjunto de consultas evaluables en ingles en este caso, $G(q)$ el conjunto de traducciones de referencia para una consulta, en este caso, en español. Definamos también q, como una palabra dentro del conjunto Q de consulta y $R_K(q)$ la lista ordenada de las primeras K traducciones predichas.

Hit@K mide la proporción de consultas en las que al menos una traducción de referencia aparece entre las primeras K predicciones:

$$\text{Hit@K} = \frac{1}{|Q|} \sum_{q \in Q} \mathbb{1} \left[R_K(q) \cap G(q) \neq \emptyset \right].$$

Mean Reciprocal Rank (MRR@K) mide el promedio del inverso del rango de la primera traducción correcta. Si no se encuentra una traducción de referencia entre las primeras K predicciones, la contribución de la consulta es cero:

$$\text{MRR@K} = \frac{1}{|Q|} \sum_{q \in Q} \begin{cases} 1/r_q^*, & \text{si existe una coincidencia en Top-}K, \\ 0, & \text{en otro caso,} \end{cases}$$

donde r_q^* es la posición de la primera traducción correcta.

$$r_q^* = \min\{i \geq 1 : r_i \in G(q)\}$$

Ambas métricas toman valores en $[0, 1]$. Hit@K evalúa la presencia de una traducción correcta en las primeras posiciones, mientras que MRR@K incorpora información sobre su posición.

En el sistema propuesto, estas métricas se utilizan para comparar la búsqueda exhaustiva por similitud coseno con la búsqueda aproximada mediante Locality-Sensitive Hashing (LSH). Esta comparación permite estudiar el compromiso entre calidad de recuperación y eficiencia computacional.

9.1 LSH

Este método, permite optimizar la busqueda, en vez de exhaustivamente, calcular una similitud coseno y ordenar con respecto a todos los valores de vectores para encontrar aquel vector más cercano con el que se busca la traducción. Este toma subconjuntos de vectores para poder hacer caclular los candidatos y sus similitudes coseno

$$N \to |C(q)| \to TopK(q)$$

Donde definiremos a $C(q)$, como el subconjunto de candidatos que toma LSH. Existe una variante del LSH, denominada *random hyperplane LSH*, donde

$$h_a(x) = \begin{cases} 1 & \text{si} a^T x \geq 0 \\ 1 & \text{si} a^T x < 0 \end{cases}$$

Donde $a \in \mathbb{R}^d$ es un vector normal con respecto a x que indicara la dirección del vector. Con esto, podemos producir una firma en la región. Como tenemos 1000 dimensiones por el embedding y 12 de los hiperplanos que aunque este valor es un hiperpárametro, inicialmente empezcamos con esto, entonces, las firmas están dadas por

$$z = x^T A$$

Donde $A \in \mathbb{R}^{d \times b}$ siendo $d = 1000$ y $b = 12$ Por lo tanto $H(z) = (1[z_1 \geq 0, \ldots, z_b \geq 0])$ ESto implica, que cada vector podra tener 12 bits de firmas, implicando que puede tener $2^{12} = 4096$ codigos diferentes. Ahora, podemos definir la similitud coseno por una propiedad, en la que si tenemos dos vectores x, y, entonces la probabildidad de que un hiperplano se asigne al mismo bit para ambos vectores

$$p(s) = 1 - \frac{\arccos(s)}{\pi}$$

Donde s es $s = cos(\theta)$ y π, representa la normalizacion de la distancia coseno. Como se observa, el problema de asignacion termina siendo probaiblsitico, asi que para mejorar la precision del valor que pueda pertenecer a cierto hiperplanos, entonces hacemos un calculo de varias *tablas*. Se proppone que la busqueda de probabildiad que cierta distancia coseno pertenezca cierto hiperplano se define como $p^b(s)$ de hiperplanos, pero para encontrar mayor exactitud, definimos L tablas independientes que

$$P_{\text{recuperación}} = 1 - (1 - p_b)^L$$

Abajo se evaluaran y comparan ambos metodos, el exhaustivo y LSH, a su vez, con se evaluan con las metricas propuestas.

[28]:
```
from time import perf_counter
import numpy as np
from sklearn.preprocessing import normalize
class TranslatorBilingual:
    DIRECTIONS = ("en-es", "es-en")
    def __init__(self, embeddings_input, vocabulary_es, vocabulary_en):
        self.vocabulary_es = vocabulary_es
        self.vocabulary_en = vocabulary_en
        self.en_embeddings = None
        self.es_embeddings = None
        self.metadata = None
        self.en_ids = None
        self.es_ids = None
        self.lsh_indices = {}
        self.load_embeddings(embeddings_input)
        self.prepare_embeddings()
    # ------------------------------------------------------------
    # 1. Carga y preparación de embeddings
    # ------------------------------------------------------------
```

```
    def load_embeddings(self, src):
        with np.load(f"{src}/metadata.npz",allow_pickle=False) as metadata:
            self.en_ids = metadata["en_ids"].copy()
            self.es_ids = metadata["es_ids"].copy()
        self.en_embeddings = np.load(f"{src}/en_embeddings.
↪npy",allow_pickle=False)
        self.es_embeddings = np.load(f"{src}/es_embeddings.
↪npy",allow_pickle=False)
    def prepare_embeddings(self):
        if (self.en_embeddings is None or self.es_embeddings is None or self.
↪en_ids is None or self.es_ids is None ):
            raise ValueError("Los embeddings o sus metadatos no se han cargado.
↪")
        if len(self.en_ids) != len(self.en_embeddings):
            raise ValueError("Los IDs ingleses no coinciden con los embeddings.
↪")
        if len(self.es_ids) != len(self.es_embeddings):
            raise ValueError("Los IDs españoles no coinciden con los embeddings.
↪")
        if self.en_embeddings.ndim != 2 or self.es_embeddings.ndim != 2:
            raise ValueError("Los embeddings deben ser matrices 2D.")
        if self.en_embeddings.shape[1] != self.es_embeddings.shape[1]:
            raise ValueError("Los embeddings deben tener la misma␣
↪dimensionalidad.")
        if not np.isfinite(self.en_embeddings).all():
            raise ValueError("Los embeddings ingleses contienen valores no␣
↪finitos.")
        if not np.isfinite(self.es_embeddings).all():
            raise ValueError("Los embeddings españoles contienen valores no␣
↪finitos.")

        # Normalización L2 para calcular similitud coseno.
        self.en_norm = normalize(self.en_embeddings, norm="l2").astype(np.
↪float32, copy=False)
        self.es_norm = normalize(self.es_embeddings, norm="l2").astype(np.
↪float32, copy=False)
        # Relación entre ID del vocabulario y fila de la matriz.
        self.en_id_to_row = {
            int(token_id): i
            for i, token_id in enumerate(self.en_ids)
        }
        self.es_id_to_row = {
            int(token_id): i
            for i, token_id in enumerate(self.es_ids)
        }
        if len(self.en_id_to_row) != len(self.en_ids):
```

```
            raise ValueError("Hay IDs ingleses duplicados.")
        if len(self.es_id_to_row) != len(self.es_ids):
            raise ValueError("Hay IDs españoles duplicados.")
    # ----------------------------------------------------------
    # 2. Configuración de la dirección
    # ----------------------------------------------------------
    def _get_direction_data(self, direction):
        if direction == "en-es":
            return {
                "source_vocab": self.vocabulary_en,
                "target_vocab": self.vocabulary_es,
                "source_norm": self.en_norm,
                "target_norm": self.es_norm,
                "source_id_to_row": self.en_id_to_row,
                "target_ids": self.es_ids,
            }

        if direction == "es-en":
            return {
                "source_vocab": self.vocabulary_es,
                "target_vocab": self.vocabulary_en,
                "source_norm": self.es_norm,
                "target_norm": self.en_norm,
                "source_id_to_row": self.es_id_to_row,
                "target_ids": self.en_ids,
            }

        raise ValueError(
            "direction debe ser 'en-es' o 'es-en'."
        )

    def _get_source_row(self, token, direction):
        data = self._get_direction_data(direction)
        vocab = data["source_vocab"]
        token_id = vocab.get_id(token)
        if token_id in (vocab.unk_id, vocab.pad_id):
            return None
        return data["source_id_to_row"].get(int(token_id))
    # ----------------------------------------------------------
    # 3. Recuperación exhaustiva
    # ----------------------------------------------------------
    def get_topk_translations(self,token,k=10,direction="en-es"):
        if k <= 0:
            return []
        data = self._get_direction_data(direction)
        source_row = self._get_source_row(token, direction)
        if source_row is None:
```

```
            return []
        source_vector = data["source_norm"][source_row]
        target_norm = data["target_norm"]
        target_ids = data["target_ids"]
        target_vocab = data["target_vocab"]
        # Similitud coseno entre el origen y todos los destinos.
        scores = target_norm @ source_vector
        k = min(k, len(scores))
        if k == 0:
            return []
        top_indices = np.argpartition(scores, len(scores) - k)[-k:]
        top_indices = top_indices[np.argsort(scores[top_indices])[::-1]]
        return [(target_vocab.get_token(int(target_ids[j])),float(scores[j]))␣
↪for j in top_indices]
    # ------------------------------------------------------------
    # 4. Hash LSH y construcción de índices
    # ------------------------------------------------------------
    @staticmethod
    def _lsh_hash(vector, planes):
        bits = vector @ planes >= 0
        code = 0
        for i, bit in enumerate(bits):
            if bit:
                code |= 1 << i
        return code

    def fit_lsh(self,direction="en-es",n_tables=10,n_planes=12,random_state=42):
        data = self._get_direction_data(direction)
        if not 1 <= n_planes <= 63:
            raise ValueError("n_planes debe estar entre 1 y 63.")
        if n_tables <= 0:
            raise ValueError("n_tables debe ser positivo.")
        target_norm = data["target_norm"]
        dimensions = target_norm.shape[1]
        rng = np.random.default_rng(random_state)
        planes_list = []
        tables_list = []
        for _ in range(n_tables):
            planes = rng.normal(size=(dimensions, n_planes)).astype(np.float32)
            table = {}
            for row, vector in enumerate(target_norm):
                code = self._lsh_hash(vector, planes)
                table.setdefault(code, []).append(row)
            planes_list.append(planes)
            tables_list.append(table)
        self.lsh_indices[direction] = {
            "planes": planes_list,
```

```
            "tables": tables_list,
            "n_tables": n_tables,
            "n_planes": n_planes,
        }

        return self

    def _get_lsh_candidate_rows(self, source_row, direction="en-es"):
        if direction not in self.lsh_indices:
            raise RuntimeError(f"Primero ejecuta␣
↪fit_lsh(direction='{direction}').")
        data = self._get_direction_data(direction)
        index = self.lsh_indices[direction]
        query = data["source_norm"][source_row]
        candidates = set()
        for planes, table in zip(index["planes"], index["tables"]):
            code = self._lsh_hash(query, planes)
            candidates.update(table.get(code, []))
        return sorted(candidates)
    # ------------------------------------------------------------
    # 5. Recuperación LSH
    # ------------------------------------------------------------
    def get_topk_lsh(self,token,k=10,direction="en-es"):
        if k <= 0:
            return []
        if direction not in self.lsh_indices:
            raise RuntimeError(f"Primero ejecuta␣
↪fit_lsh(direction='{direction}').")
        data = self._get_direction_data(direction)
        source_row = self._get_source_row(token, direction)
        if source_row is None:
            return []
        candidate_rows = self._get_lsh_candidate_rows(source_row, direction)
        if not candidate_rows:
            return []
        candidate_rows = np.asarray(candidate_rows, dtype=int)
        source_vector = data["source_norm"][source_row]
        target_norm = data["target_norm"]
        target_ids = data["target_ids"]
        target_vocab = data["target_vocab"]
        scores = target_norm[candidate_rows] @ source_vector
        k = min(k, len(scores))
        top_positions = np.argpartition(scores, len(scores) - k)[-k:]
        top_positions = top_positions[np.argsort(scores[top_positions])[::-1]]
        return [
            (target_vocab.
↪get_token(int(target_ids[candidate_rows[pos]])),float(scores[pos]))
```

```
            for pos in top_positions
        ]
    # ------------------------------------------------------------
    # 6. Evaluación Hit@K y MRR
    # ------------------------------------------------------------
    def evaluate(self,gold_translations,k_values=(1, 5,␣
↪10),method="exhaustive",direction="en-es"):
        if not k_values or any(k <= 0 for k in k_values):
            raise ValueError("Todos los valores de K deben ser positivos.")
        if len(set(k_values)) != len(k_values):
            raise ValueError("No repitas valores de K.")
        if method not in ("exhaustive", "lsh"):
            raise ValueError("method debe ser 'exhaustive' o 'lsh'.")
        if method == "lsh" and direction not in self.lsh_indices:
            raise RuntimeError(f"Construye primero el índice LSH para␣
↪'{direction}'.")
        self._get_direction_data(direction)
        hits = {k: 0 for k in k_values}
        reciprocal_ranks = []
        evaluated = 0
        skipped = []
        for source_word, gold in gold_translations.items():
            source_row = self._get_source_row(source_word, direction)
            if source_row is None:
                skipped.append(source_word)
                continue
            if isinstance(gold, str):
                gold = {gold}
            else:
                gold = set(gold)
            if not gold:
                skipped.append(source_word)
                continue
            if method == "exhaustive":
                predictions = self.
↪get_topk_translations(source_word,k=max(k_values),direction=direction)
            else:
                predictions = self.
↪get_topk_lsh(source_word,k=max(k_values),direction=direction)
            predicted_words = [word for word, _ in predictions ]
            evaluated += 1
            for k in k_values:
                if any(word in gold for word in predicted_words[:k] ):
                    hits[k] += 1
            rank = next((i for i, word in enumerate(predicted_words, start=1)␣
↪if word in gold),None)
            reciprocal_ranks.append(1.0 / rank if rank is not None else 0.0)
```

```
        if evaluated == 0:
            raise ValueError("No se pudo evaluar ninguna palabra del conjunto.")
        return {
            **{
                f"hit@{k}": hits[k] / evaluated
                for k in k_values
            },
            f"mrr@{max(k_values)}": (
                sum(reciprocal_ranks) / evaluated
            ),
            "evaluated": evaluated,
            "skipped": skipped,
        }
    # ------------------------------------------------------------
    # 7. Comparación de recuperación exhaustiva y LSH
    # ------------------------------------------------------------
    def compare_retrievers(self,gold_translations,k_values=(1, 5,␣
↪10),direction="en-es",):
        if direction not in self.lsh_indices:
            raise RuntimeError(f"Ejecuta fit_lsh(direction='{direction}') antes␣
↪de comparar.")
        data = self._get_direction_data(direction)
        valid_words = []
        skipped = []
        for word in gold_translations:
            if self._get_source_row(word, direction) is None:
                skipped.append(word)
            else:
                valid_words.append(word)
        if not valid_words:
            raise ValueError("No hay palabras válidas para realizar la␣
↪comparación.")
        exhaustive_times = []
        lsh_times = []
        candidate_counts = []
        max_k = max(k_values)
        for word in valid_words:
            start = perf_counter()
            self.get_topk_translations(word, k=max_k, direction=direction)
            exhaustive_times.append(perf_counter() - start)
            source_row = self._get_source_row(word, direction)
            start = perf_counter()
            self.get_topk_lsh(word, k=max_k, direction=direction)
            lsh_times.append(perf_counter() - start)
            candidates = self._get_lsh_candidate_rows(source_row, direction)
            candidate_counts.append(len(candidates))
```

```
        exhaustive_metrics = self.
↪evaluate(gold_translations,k_values=k_values,method="exhaustive",direction=direction)
        lsh_metrics = self.
↪evaluate(gold_translations,k_values=k_values,method="lsh",direction=direction)
        exhaustive_ms = (np.mean(exhaustive_times) * 1000)
        lsh_ms = np.mean(lsh_times) * 1000
        mean_candidates = np.mean(candidate_counts)
        return {
            "direction": direction,
            "queries": len(valid_words),
            "skipped": skipped,
            "exhaustive": {
                "metrics": exhaustive_metrics,
                "mean_latency_ms": float(exhaustive_ms),
            },
            "lsh": {
                "metrics": lsh_metrics,
                "mean_latency_ms": float(lsh_ms),
                "mean_candidates": float(mean_candidates),
                "candidate_ratio": (
                    float(mean_candidates / len(data["target_ids"]))
                ),
            },
            "speedup": (
                float(exhaustive_ms / lsh_ms)
                if lsh_ms > 0
                else None
            ),
        }

    # ------------------------------------------------------------
    # 8. Traducción palabra por palabra
    # ------------------------------------------------------------
    def translate_sentence(self,sentence,k=1,direction="en-es",method="lsh",):
        if method not in ("lsh", "exhaustive"):
            raise ValueError("method debe ser 'lsh' o 'exhaustive'.")
        if method == "lsh" and direction not in self.lsh_indices:
            raise RuntimeError(f"Construye primero el índice LSH para␣
↪'{direction}'.")
        source_tokens = tokenize(sentence)
        translated_tokens = []
        for token in source_tokens:
            # Conserva signos de puntuación.
            if not any(char.isalnum() for char in token):
                translated_tokens.append(token)
                continue
            if method == "lsh":
```

```
                candidates = self.get_topk_lsh(token, k=k, direction=direction)
            else:
                candidates = self.get_topk_translations(token, k=k,␣
 ↪direction=direction)
            if candidates:
                translated_tokens.append(candidates[0][0])
            else:
                # Si no hay candidato, conserva el token original.
                translated_tokens.append(token)
        return detokenize(translated_tokens)
```

[29]:
```
translator = TranslatorBilingual(
    embeddings_input="artifacts/embeddings",
    vocabulary_es=es_vocab,
    vocabulary_en=en_vocab,
)

# Validar embeddings para español → inglés
print("Embeddings ES (origen):", translator.es_embeddings.shape)
print("Embeddings EN (destino):", translator.en_embeddings.shape)

print("IDs ES:", len(translator.es_ids))
print("IDs EN:", len(translator.en_ids))

assert translator.es_embeddings.shape[0] == len(translator.es_ids)
assert translator.en_embeddings.shape[0] == len(translator.en_ids)
assert translator.es_embeddings.shape[1] == translator.en_embeddings.shape[1]

print("Carga y dimensiones correctas para ES → EN.")
```

```
Embeddings ES (origen): (238128, 1000)
Embeddings EN (destino): (185546, 1000)
IDs ES: 238128
IDs EN: 185546
Carga y dimensiones correctas para ES → EN.
```

[30]:
```
for word in ["gato", "perro", "casa", "agua", "libro"]:
    candidates = translator.get_topk_translations(word,k=5,direction="es-en")
    print(f"{word} → {candidates}")
```

```
gato → [('cat', 0.9997729659080505), ('club', 0.12267708778381348), ('j',
0.11946220695972443), ('dark', 0.11890615522861481), ('image',
0.11133147776126862)]
perro → [('dog', 0.9514172077178955), ('couple', 0.275431752204895), ('a',
0.25646278262138367), ('murder', 0.25043025612831116), ('attack',
0.23782965540885925)]
casa → [('home', 0.9087450504302979), ('house', 0.8728690147399902), ('place',
0.5478805303573608), ('her', 0.3332260251045227), ('go', 0.3176957368850708)]
```

```
agua → [('water', 0.5640203356742859), ('or', 0.5079786777496338), ('from',
0.418693482875824), ('(', 0.343877375125885), ('as', 0.34297311305999756)]
libro → [('book', 0.7002827525138855), ('this', 0.4859514832496643), ('church',
0.41114211082458496), ('commitment', 0.3900338411331177), ('is',
0.3472141623497009)]
```

[31]:
```
gold_translations = {
    "gato": {"cat"},
    "perro": {"dog"},
    "casa": {"house", "home"},
    "agua": {"water"},
    "libro": {"book"},
    "comida": {"food"},
    "alimento": {"food"},
    "coche": {"car"},
    "auto": {"car"},
    "automóvil": {"car", "automobile"},
    "ciudad": {"city"},
    "escuela": {"school"},
    "amigo": {"friend"},
    "amiga": {"friend"},
}
metrics = translator.evaluate(gold_translations,k_values=(1, 5,␣
 ↪10),direction="es-en",)
print(metrics)
```

```
{'hit@1': 0.7142857142857143, 'hit@5': 0.8571428571428571, 'hit@10':
0.8571428571428571, 'mrr@10': 0.7857142857142857, 'evaluated': 14, 'skipped':
[]}
```

[32]:
```
from time import perf_counter

start = perf_counter()

translator.fit_lsh(direction="es-en",n_tables=10,n_planes=12,random_state=42)
print(f"Construcción LSH: {perf_counter() - start:.2f} s")

start = perf_counter()
results = translator.get_topk_lsh("gato",k=10,direction="es-en")
print(f"Consulta LSH: {perf_counter() - start:.6f} s")
print(f"gato → {results}")
```

```
Construcción LSH: 8.30 s
Consulta LSH: 0.000424 s
gato → [('cat', 0.9997729659080505), ('effectiveness', 0.05942784249782562),
('certificate', 0.026997040957212448), ('indeed', 0.006438218057155609),
('done', 0.004751283675432205), ('develop', 0.004631355870515108), ('healthy',
0.0019536579493433237), ('heavy', 0.00194438430480659), ('too',
0.0012829770566895604), ('sports', 0.0011096427915617824)]
```

[33]:
```
from pprint import pprint
from time import perf_counter
start = perf_counter()
results = translator.compare_retrievers(gold_translations,k_values=(1, 5,␣
 ↪10),direction="es-en")
print(f"Comparación completada en {perf_counter() - start:.2f} s")
pprint(results)
```

```
Comparación completada en 1.53 s
{'direction': 'es-en',
 'exhaustive': {'mean_latency_ms': 30.04005528678785,
                'metrics': {'evaluated': 14,
                            'hit@1': 0.7142857142857143,
                            'hit@10': 0.8571428571428571,
                            'hit@5': 0.8571428571428571,
                            'mrr@10': 0.7857142857142857,
                            'skipped': []}},
 'lsh': {'candidate_ratio': 0.13985211214469728,
         'mean_candidates': 25949.0,
         'mean_latency_ms': 23.557140142656863,
         'metrics': {'evaluated': 14,
                     'hit@1': 0.6428571428571429,
                     'hit@10': 0.7857142857142857,
                     'hit@5': 0.7857142857142857,
                     'mrr@10': 0.7142857142857143,
                     'skipped': []}},
 'queries': 14,
 'skipped': [],
 'speedup': 1.2751995830084586}
```

[34]:
```
from time import perf_counter
import pandas as pd
configurations = [
    {"name": "base_10x12", "n_tables": 10, "n_planes": 12},
    {"name": "15x12", "n_tables": 15, "n_planes": 12},
    {"name": "20x12", "n_tables": 20, "n_planes": 12},
    {"name": "15x10", "n_tables": 15, "n_planes": 10},
    {"name": "20x10", "n_tables": 20, "n_planes": 10},
]
results = []
for config in configurations:
    print(f"\nProbando {config['name']}...")
    start = perf_counter()
    translator.
 ↪fit_lsh(direction="es-en",n_tables=config["n_tables"],n_planes=config["n_planes"],random_st
    build_seconds = perf_counter() - start
```

```
    comparison = translator.compare_retrievers(gold_translations,k_values=(1,␣
 ↪5, 10),direction="es-en")
    lsh = comparison["lsh"]
    metrics = lsh["metrics"]
    results.append({
        "config": config["name"],
        "build_seconds": build_seconds,
        "hit@1": metrics["hit@1"],
        "hit@5": metrics["hit@5"],
        "hit@10": metrics["hit@10"],
        "mrr@10": metrics["mrr@10"],
        "latency_ms": lsh["mean_latency_ms"],
        "candidates": lsh["mean_candidates"],
        "candidate_ratio": lsh["candidate_ratio"],
        "speedup": comparison["speedup"],
    })

df_results = pd.DataFrame(results)

print("\nResultados:")
print(df_results.to_string(index=False,float_format=lambda x: f"{x:.6f}"))
```

```
Probando base_10x12…

Probando 15x12…

Probando 20x12…

Probando 15x10…

Probando 20x10…

Resultados:
    config  build_seconds    hit@1    hit@5   hit@10   mrr@10  latency_ms
candidates  candidate_ratio  speedup
base_10x12       8.434408 0.642857 0.785714 0.785714 0.714286   24.021245
25949.000000         0.139852 1.322955
     15x12      12.585782 0.642857 0.785714 0.785714 0.714286   24.689061
25952.642857         0.139872 1.122514
     20x12      16.744774 0.642857 0.785714 0.785714 0.714286   26.439771
25956.142857         0.139891 1.093266
     15x10      13.565539 0.642857 0.785714 0.785714 0.714286   35.136951
38955.571429         0.209951 0.822727
     20x10      18.082525 0.642857 0.785714 0.785714 0.714286   36.271431
38972.571429         0.210043 0.768248
```

10 Resultados

Podemos observar que LSH es mucho más rapido que el método exhaustivo y aunque tiene cierta bajada de hit y MRR, no resulta ser tan fuerte como para abandor la idea uso, ya que es un método mucho más rápido y menos pesado computacionalmente # Evaluación final Finalmente, se van a evaluar este modelo con el conjunto de validación evaluación, estos conjuntos ya los tenemos precargados en csv por los preprocesamientos anterior, y aquí sería valioso poder utilizar otras métricas mucho más robustas que hit y mrr, como BLEU siendo un ejemplo Para esto, se añade en la clase del modelo, un modulo para traducir ya no una plabra y sus candidatos sino una oracion completa

[35]:
```
words = [
    "el", "gato", "bebe", "agua",
    "yo", "tengo", "un", "libro",
    "perro", "está", "en", "casa",
    "nosotros", "vamos", "a", "la", "escuela",
]

for word in words:
    print(f"\n{'=' * 45}")
    print(f"ES: {word}")

    if word not in translator.vocabulary_es.token_to_id:
        print("No está en el vocabulario de español")
        continue

    print("Exhaustive:")
    print(translator.get_topk_translations(word, k=5, direction="es-en"))
    print("LSH:")
    print(translator.get_topk_lsh(word, k=5, direction="es-en"))
```

```
=============================================
ES: el
Exhaustive:
[('won', 0.7186280488967896), ('weather', 0.6890209913253784), ('delivered',
0.6758937835693359), ('chosen', 0.6748637557029724), ('worldwide',
0.6706429719924927)]
LSH:
[('won', 0.7186280488967896), ('weather', 0.6890209913253784), ('chosen',
0.6748637557029724), ('match', 0.66754549741745), ('closed',
0.6616758108139038)]

=============================================
ES: gato
Exhaustive:
[('cat', 0.9997729659080505), ('club', 0.12267708778381348), ('j',
0.11946220695972443), ('dark', 0.11890615522861481), ('image',
```

```
0.11133147776126862)]
LSH:
[('cat', 0.9997729659080505), ('dark', 0.11890615522861481), ('leader',
0.08760813623666763), ('glad', 0.044651199132204056), ('occupation',
0.03260594606399536)]

============================================
ES: bebe
Exhaustive:
[('.', 0.0), (' ', 0.0), (' ', 0.0), (' ', 0.0), (' ', 0.0)]
LSH:
[('who', 0.0), (' ', 0.0), (' ', 0.0), (' ', 0.0), (' ', 0.0)]

============================================
ES: agua
Exhaustive:
[('water', 0.5640203356742859), ('or', 0.5079786777496338), ('from',
0.418693482875824), ('(', 0.343877375125885), ('as', 0.34297311305999756)]
LSH:
[('from', 0.418693482875824), ('with', 0.30444425344467163), ('representatives',
0.17400676012039185), ('prevention', 0.10344335436820984), ('less',
0.08779045194387436)]

============================================
ES: yo
Exhaustive:
[('am', 0.5835724472999573), ('myself', 0.525234580039978), ("i'd",
0.4852410554885864), ("didn't", 0.4221733808517456), ("i'll",
0.4145720303058624)]
LSH:
[('am', 0.5835724472999573), ("i'm", 0.36179542541503906), ('too',
0.294592946767807), ('do', 0.17253556847572327), ('think', 0.16812238097190857)]

============================================
ES: tengo
Exhaustive:
[('gotta', 0.8450294733047485), ('got', 0.8064800500869751), ('idea',
0.5534031987190247), ("i've", 0.4772700071334839), ('have', 0.4195006489753723)]
LSH:
[('gotta', 0.8450294733047485), ('got', 0.8064800500869751), ('am',
0.39341604709625244), ("we've", 0.320846825838089), ('get', 0.2861190438270569)]

============================================
ES: un
Exhaustive:
[('bunch', 0.8295688629150391), ('shot', 0.7692914605140686), ('hit',
0.7592434883117676), ('named', 0.7442324161529541), ('break',
0.7387714385986328)]
```

```
LSH:
[('bunch', 0.8295688629150391), ('shot', 0.7692914605140686), ('hit',
0.7592434883117676), ('named', 0.7442324161529541), ('break',
0.7387714385986328)]

==========================================
ES: libro
Exhaustive:
[('book', 0.7002827525138855), ('this', 0.4859514832496643), ('church',
0.41114211082458496), ('commitment', 0.3900338411331177), ('is',
0.3472141623497009)]
LSH:
[('book', 0.7002827525138855), ('commitment', 0.3900338411331177), ('is',
0.3472141623497009), ('the', 0.24110174179077148), ('of', 0.231790229678154)]

==========================================
ES: perro
Exhaustive:
[('dog', 0.9514172077178955), ('couple', 0.275431752204895), ('a',
0.25646278262138367), ('murder', 0.25043025612831116), ('attack',
0.23782965540885925)]
LSH:
[('dog', 0.9514172077178955), ('master', 0.13675832748413086), ('pleasure',
0.11442539840936661), ('sweden', 0.09889397770166397), ('efficiency',
0.06704846024513245)]

==========================================
ES: está
Exhaustive:
[('happening', 0.8866418600082397), ("where's", 0.7067940831184387), ('getting',
0.6537737846374512), ('located', 0.6363774538040161), ('fine',
0.6275831460952759)]
LSH:
[('happening', 0.8866418600082397), ('getting', 0.6537737846374512), ("he's",
0.6263320446014404), ("she's", 0.6085636019706726), ("who's",
0.558738112449646)]

==========================================
ES: en
Exhaustive:
[('en', 0.7722009420394897), ('sitting', 0.7605725526809692), ('enter',
0.7488873600959778), ('spent', 0.7262648344039917), ('released',
0.7156670093536377)]
LSH:
[('en', 0.7722009420394897), ('sitting', 0.7605725526809692), ('enter',
0.7488873600959778), ('spent', 0.7262648344039917), ('released',
0.7156670093536377)]
```

```
==========================================
ES: casa
Exhaustive:
[('home', 0.9087450504302979), ('house', 0.8728690147399902), ('place',
0.5478805303573608), ('her', 0.3332260251045227), ('go', 0.3176957368850708)]
LSH:
[('home', 0.9087450504302979), ('house', 0.8728690147399902), ('place',
0.5478805303573608), ('our', 0.15327021479606628), ('if', 0.09388798475265503)]

==========================================
ES: nosotros
Exhaustive:
[('us', 0.8102000951766968), ('ourselves', 0.8058955073356628), ("we're",
0.6416323184967041), ('our', 0.5443152189254761), ("we'll", 0.4695260226726532)]
LSH:
[('us', 0.8102000951766968), ('ourselves', 0.8058955073356628), ("we're",
0.6416323184967041), ('our', 0.5443152189254761), ('we', 0.30007171630859375)]

==========================================
ES: vamos
Exhaustive:
[("let's", 0.9034616947174072), ("we'll", 0.7300738096237183), ("we're",
0.6542218923568726), ('come', 0.6366243362426758), ('gonna',
0.5203712582588196)]
LSH:
[("let's", 0.9034616947174072), ("we'll", 0.7300738096237183), ('gonna',
0.5203712582588196), ('going', 0.5072584748268127), ('all',
0.03431248664855957)]

==========================================
ES: a
Exhaustive:
[('beat', 0.8108366131782532), ('shoot', 0.8091304302215576), ('catch',
0.795843243598938), ('join', 0.7934144735336304), ('calling',
0.7770466208457947)]
LSH:
[('beat', 0.8108366131782532), ('shoot', 0.8091304302215576), ('catch',
0.795843243598938), ('join', 0.7934144735336304), ('calling',
0.7770466208457947)]

==========================================
ES: la
Exhaustive:
[('ball', 0.7810966968536377), ('throw', 0.7527045011520386), ('corner',
0.7453254461288452), ('grave', 0.7449917793273926), ('heading',
0.730114221572876)]
LSH:
[('ball', 0.7810966968536377), ('throw', 0.7527045011520386), ('corner',
```

```
0.7453254461288452), ('grave', 0.7449917793273926), ('heading',
0.730114221572876)]

==========================================
ES: escuela
Exhaustive:
[('school', 0.9748525619506836), ('at', 0.23860490322113037), ('minimum',
0.12088531255722046), ('hotel', 0.09865064918994904), ('scale',
0.09558521211147308)]
LSH:
[('school', 0.9748525619506836), ('in', 0.08219228684902191), ('planning',
0.05270346626639366), ('sweden', 0.03620752692222595), ('judicial',
0.03497479110956192)]
```

[36]:
```python
import numpy as np
def inspect_token(translator, token):
    print(f"\nToken: {token!r}")
    # Comprueba el vocabulario de origen: español.
    token_id = translator.vocabulary_es.token_to_id.get(token)
    if token_id is None:
        print("No existe en el vocabulario español.")
        return
    row = translator.es_id_to_row.get(token_id)
    if row is None:
        print(f"El ID {token_id} no tiene fila de embedding.")
        return
    vector = translator.es_embeddings[row]
    norm = np.linalg.norm(vector)
    print("ID:", token_id)
    print("Fila:", row)
    print("Dimensiones:", vector.shape)
    print("Norma:", float(norm))
    print("Valores finitos:", bool(np.isfinite(vector).all()))
    print("Vector nulo:", bool(np.allclose(vector, 0)))
    print("Top 5:",translator.
 ↪get_topk_translations(token,k=5,direction="es-en",))


for word in ["el", "yo", "gato", "bebe", "agua", "está", "escuela"]:
    inspect_token(translator, word)
```

```
Token: 'el'
ID: 7
Fila: 5
Dimensiones: (1000,)
Norma: 4.9046807289123535
Valores finitos: True
```

```
Vector nulo: False
Top 5: [('won', 0.7186280488967896), ('weather', 0.6890209913253784),
('delivered', 0.6758937835693359), ('chosen', 0.6748637557029724), ('worldwide',
0.6706429719924927)]

Token: 'yo'
ID: 50
Fila: 48
Dimensiones: (1000,)
Norma: 1.563071846961975
Valores finitos: True
Vector nulo: False
Top 5: [('am', 0.5835724472999573), ('myself', 0.525234580039978), ("i'd",
0.4852410554885864), ("didn't", 0.4221733808517456), ("i'll",
0.4145720303058624)]

Token: 'gato'
ID: 3636
Fila: 3634
Dimensiones: (1000,)
Norma: 2.067445993423462
Valores finitos: True
Vector nulo: False
Top 5: [('cat', 0.9997729659080505), ('club', 0.12267708778381348), ('j',
0.11946220695972443), ('dark', 0.11890615522861481), ('image',
0.11133147776126862)]

Token: 'bebe'
ID: 4346
Fila: 4344
Dimensiones: (1000,)
Norma: 0.0
Valores finitos: True
Vector nulo: True
Top 5: [('.', 0.0), (' ', 0.0), (' ', 0.0), (' ', 0.0), (' ', 0.0)]

Token: 'agua'
ID: 353
Fila: 351
Dimensiones: (1000,)
Norma: 0.5831353664398193
Valores finitos: True
Vector nulo: False
Top 5: [('water', 0.5640203356742859), ('or', 0.5079786777496338), ('from',
0.418693482875824), ('(', 0.343877375125885), ('as', 0.34297311305999756)]

Token: 'está'
ID: 43
```

```
Fila: 41
Dimensiones: (1000,)
Norma: 2.3240435123443604
Valores finitos: True
Vector nulo: False
Top 5: [('happening', 0.8866418600082397), ("where's", 0.7067940831184387),
('getting', 0.6537737846374512), ('located', 0.6363774538040161), ('fine',
0.6275831460952759)]

Token: 'escuela'
ID: 685
Fila: 683
Dimensiones: (1000,)
Norma: 1.8300482034683228
Valores finitos: True
Vector nulo: False
Top 5: [('school', 0.9748525619506836), ('at', 0.23860490322113037), ('minimum',
0.12088531255722046), ('hotel', 0.09865064918994904), ('scale',
0.09558521211147308)]
```

[37]:
```python
import sacrebleu
from time import perf_counter
def evaluate_sentence_translation(translator, processed_df, split_name):
    sources = processed_df["es_norm"].fillna("").tolist()
    references = processed_df["en_norm"].fillna("").tolist()
    start = perf_counter()
    hypotheses = [translator.
 ↪translate_sentence(sentence,k=1,direction="es-en",method="lsh",)for sentence␣
 ↪in sources]
    elapsed = perf_counter() - start
    bleu = sacrebleu.corpus_bleu(hypotheses,[references],lowercase=True)
    chrf = sacrebleu.corpus_chrf(hypotheses,[references])
    print(f"\n--- {split_name} (ES → EN) ---")
    print(f"Oraciones: {len(sources)}")
    print(f"BLEU:      {bleu.score:.2f}")
    print(f"chrF:      {chrf.score:.2f}")
    print(f"Tiempo:    {elapsed:.2f} s")
    print(f"Promedio:  "f"{elapsed / max(len(sources), 1) * 1000:.2f} ms/
 ↪oración")
    return {
        "split": split_name,
        "n": len(sources),
        "bleu": bleu.score,
        "chrf": chrf.score,
        "seconds": elapsed,
        "hypotheses": hypotheses,
    }
```

[38]:
```
validation_results =␣
↪evaluate_sentence_translation(translator,validation_processed,"Validation")
```

```
--- Validation (ES → EN) ---
Oraciones: 1998
BLEU:      3.51
chrF:      27.15
Tiempo:    802.78 s
Promedio:  401.79 ms/oración
```

[39]:
```
test_results = evaluate_sentence_translation(translator,test_processed,"Test")
```

```
--- Test (ES → EN) ---
Oraciones: 1996
BLEU:      3.02
chrF:      27.27
Tiempo:    806.44 s
Promedio:  404.03 ms/oración
```

11 Metricas finales

Para realizar una traducion a nivel de oracion, se necesitan ahora metricas como BLEU y chrf que permiten evalauir que tan bien tradujo una oracion ## BLEU BLEU (Bilingual Evaluation Understudy) mide cuánto coincide la traducción generada con una o varias traducciones de referencia, por medio de los n grams, se calcula de la siguinete manera

$$\text{BLEU} = BP \cdot \exp\left(\sum_{n=1}^{N} w_n \log p_n\right)$$

Donde: - p_n: precisión modificada de los n-gramas de tamaño n. - w_n: peso asignado a cada tamaño; normalmente 1/4 para n-gramas de 1 a 4 palabras. - BP: penalización por brevedad (brevity penalty). La penalización es:

$$BP = \begin{cases} 1 & c > r \\ e^{1-r/c} & 0 < c \leq r \\ 0 & c = 0 \end{cases}$$

Aquí c es la longitud total de las traducciones generadas y r la longitud de referencia correspondiente. ESto se puede entender como que BLEU premia que las palabras y sus secuencias coincidan con la referencia, pero puede penalizar paráfrasis válidas que usen otras palabras.

Por esta razón, en métricas de evaluación de traductores no solamente se cuenta métrica, sino también METEOR o chrf # chrF: coincidencia de secuencias de caracteres chrF (Character n-gram F-score) compara las traducciones a nivel de caracteres, en lugar de depender exclusivamente de palabras completas. Esto permite reconocer similitudes parciales, como palabras con la misma raíz, terminaciones similares o pequeñas diferencias ortográficas. Aquí si permite o es más flexible que

BLEU. El como se calcula esta métrica es por medio de la obtención de n grams de la hipotesis y de la referencia . ESte calcula la precision y el recall a nivel caracter, y un f_β. Por lo general, la libreria calcula un F_2, dando mas penalizacion al presicion y potenciando el recall.

Como tal, no hay valores que indiquen que tan bien o mal, aunque estas métricas dependen del dataset, pero, por lo general si se obtienen valores muy cerca de 0, es que no hay muchas coincidencias, mientras que valores más arriba de 50-60 es que poseen muchas coincidencias # Conclusiones finales

Este modelo sirve como base principal para poder ser comparado con próximos modelos, este modelo traduce palabras de forma correcta pero se le dificulta en la parte de traducir frases completas. Y esto es parte de las complejidades linguisticas, donde se necesita ademas de la coocurrencia, una metodologia para poder encontrar la relación o contexto global dentro de una oración, frase o párrafo

[ ]: